



VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

BRNO UNIVERSITY OF TECHNOLOGY

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

FACULTY OF INFORMATION TECHNOLOGY

ÚSTAV INFORMAČNÍCH SYSTÉMŮ

DEPARTMENT OF INFORMATION SYSTEMS

ŘEŠENÍ FUTOSHIKI POMOCÍ AUTOMATŮ

THESIS TITLE

BAKALÁŘSKÁ PRÁCE

BACHELOR'S THESIS

AUTOR PRÁCE

AUTHOR

MARTIN SLEZÁK

VEDOUCÍ PRÁCE

SUPERVISOR

prof. RNDr. ALEXANDER MEDUNA, CSc.

BRNO 2025

Abstrakt

Cílem této práce je návrh automatu pro řešení hlavolamu Futoshiki a analýza efektivity tohoto přístupu v porovnání s tradičními metodami, s následnou implementací řešitele. Pro řešení problému byly navrženy dvoudimenzionální automaty využívající několik algoritmů, které zajišťují efektivní a spolehlivé řešení. Implementace automatů je součástí aplikace, která umožňuje vytvářet hrací pole, generovat náhodné pole, hrát hru manuálně, nebo využít automatické řešení pomocí navržených automatů.

Abstract

The aim of this thesis is to design an automaton for solving the Futoshiki puzzle and to analyze the efficiency of this approach in comparison to traditional methods, followed by the implementation of a solver. The chosen problem is addressed through the development of two-dimensional automata that utilize various algorithms, providing an efficient and reliable solution. The implemented automata are part of an application that allows users to create custom game grids, generate random grid, play the game manually, or utilize automated solving through the designed automata.

Klíčová slova

Futoshiki, automat, dvoudimenzionální automaty, backtracking, forward checking, čtyřcestný konečný automat

Keywords

Futoshiki, automaton, two-dimensional automata, backtracking, forward checking, four-way finite automata

Citace

SLEZÁK, Martin. *Řešení Futoshiki pomocí automatů*. Brno, 2025. Bakalářská práce. Vysoké učení technické v Brně, Fakulta informačních technologií. Vedoucí práce prof. RNDr. Alexander Meduna, CSc.

Řešení Futoshiki pomocí automatů

Prohlášení

Prohlašuji, že jsem tuto bakalářskou práci vypracoval samostatně pod vedením prof. RNDr. Alexandra Meduny, CSc. Uvedl jsem všechny literární prameny, publikace a další zdroje, ze kterých jsem čerpal.

.....
Martin Slezák
10. března 2025

Poděkování

Rád bych poděkoval svému vedoucímu, prof. RNDr. Alexandrovi Medunovi, CSc., za jeho cenné rady, odborné vedení a ochotu při konzultacích. Jeho vstřícný a profesionální přístup byl pro mě velkou inspirací během celé práce.

Obsah

1	Úvod	4
2	Futoshiki a algoritmy pro řešení	5
2.1	Pravidla hry	5
2.2	Backtracking	5
2.2.1	Backtracking v praxi	6
2.2.2	Backtracking a Futoshiki	6
2.3	Forward Checking	6
2.3.1	Heuristiky Forward Checkingu	6
2.3.2	Variace Forward Checkingu	7
2.3.3	Forward Checking a Futoshiki	7
2.4	Arc Consistency #3	8
2.4.1	AC-3 a Futoshiki	8
3	Existující formy konečných automatů	9
3.1	Konečný automat	9
3.1.1	Deterministický konečný automat	9
3.1.2	Nedeterministický konečný automat	9
3.1.3	Dvojcestný konečný automat	10
3.1.4	Skákající automaty	10
3.1.5	Skákající automat s opakovaným čtením	10
4	Dvojdímenzionální automaty	12
4.1	Dvojdímenzionální skákající automat	12
5	Pravidla dvojdímenzionálního skákajícího automatu	13
5.1	Základní značení	13
5.2	Vytvoření domén polí	14
5.3	Řešení hry	14
6	Implementace	15
6.1	Spuštění aplikace	15
6.2	Vzhled a možnosti aplikace	16
6.2.1	Vytváření mřížky	17
6.2.2	Řešení mřížky	17
6.2.3	Změna řešícího algoritmu	18
7	Testování	19

7.1	Správnost řešení	19
7.2	Časová náročnost	20
7.2.1	Zhodnocení	20
8	Závěr	22
	Literatura	23
A	Obsah CD	24

Seznam obrázků

6.1	Vzhled aplikace po spuštění hry se světlým tématem	16
6.2	Stránka výběru řešícího algoritmu	18
7.1	Srovnání časové náročnosti všech algoritmů při různých velikostech mřížky (4×4 až 10×10).	20

Kapitola 1

Úvod

V této práci se čtenář dozví informace o hře Futoshiki, její pravidla a různých metodách, které slouží pro řešení této hry. Čtenář se dozví o konečných automatech, které budou využity pro vytovření automatu řešitele hlavolamu.

Kapitola 2

Futoshiki a algoritmy pro řešení

Tato kapitola se podrobně zaměřuje na pravidla hry Futoshiki a algoritmy, které jsou používány k jejímu řešení. Nejprve se seznámíme s pravidly, která jsou nezbytná pro pochopení herních mechanismů, což nám umožní lépe porozumět následně popsáným algoritmům pro řešení hlavolamu.

Prvním a základním algoritmem, který budeme zkoumat, je algoritmus Backtracking. Tento algoritmus využívá princip systematického prozkoumávání všech možných řešení metodou pokus-omyl. Ačkoli není nejefektivnější, poskytuje klíčový základ pro pochopení dalších sofistikovanějších metod, které lze využít k řešení hlavolamu Futoshiki.

Následně se zaměříme na algoritmus Forward Checking a jeho dvě možné implementace. Tento algoritmus vylepšuje základní princip Backtrackingu tím, že předběžně kontroluje možné hodnoty, což vede ke snížení počtu zkoušených kombinací.

Všechny popsané algoritmy byly implementovány v programu, který umožňuje řešení libovolné mřížky hlavolamu Futoshiki. Tato implementace nám poskytuje příležitost demonstrovat účinnost jednotlivých algoritmů v různých herních scénářích.

2.1 Pravidla hry

Futoshiki [3], známá také jako hra "větší/menší" (More or Less), je logický hlavolam pocházející z Japonska, který je založen na podobném principu jako sudoku. Hlavolam se skládá ze čtvercové mřížky, obvykle o velikosti 4x4, přičemž může být i větší. Pro úspěšné vyřešení je třeba každé pole v mřížce vyplnit číslem od 1 do velikosti mřížky. Dále musí platit pravidlo, že v každém řádku a sloupci se čísla nesmí opakovat. Kromě toho jsou na mřížce umístěny nerovnosti mezi některými poli, které určují vztah mezi dvěma čísly. Hra začíná s libovolným počtem již vyplněných polí a nerovností, přičemž toto počáteční rozmístění musí být v souladu s pravidly a zaručovat existenci alespoň jednoho řešení. Následně hráč vyplňuje zbývajících pole, zatímco podmínky se již nemění a nepřidávají se žádné nové nerovnosti.

2.2 Backtracking

Backtracking [2] je jednoduchý a často využívaný algoritmus pro řešení problémů, které vyžadují prozkoumání velkého množství možných řešení. Algoritmus pracuje na principu systematického prohledávání prostoru řešení metodou pokus-omyl. Proces začíná s jednou možnou cestou a postupuje do hloubky. Pokud aktuálně prohledávaná cesta nesplňuje zadané podmínky (například porušuje pravidla problému), algoritmus se vrací zpět k před-

chozímu bodu rozhodnutí a zkouší jinou možnost. Tento cyklus se opakuje, dokud není nalezeno řešení splňující všechny podmínky, nebo dokud nejsou vyčerpány všechny možné cesty.

2.2.1 Backtracking v praxi

Algoritmus backtracking se vyznačuje jednoduchou implementací, což však přináší i určité nevýhody. Mezi hlavní slabiny patří neefektivita při prohledávání prostoru řešení - algoritmus zkouší všechny možnosti, včetně těch, které je zjevně možné vyloučit již na počátku. Tento nedostatek způsobuje, že backtracking je vhodný především pro řešení problémů menšího rozsahu, jako je například mřížka Futoshiki o velikosti 4×4 .

S rostoucí velikostí mřížky se však počet možných kombinací exponenciálně zvyšuje, což vede k rapidnímu poklesu efektivity. Pro větší problémy se proto backtracking obvykle kombinuje s dalšími technikami, které umožňují zúžení prostoru řešení a tím i zlepšení výpočetní náročnosti.

2.2.2 Backtracking a Futoshiki

Algoritmus backtracking při řešení hlavolamu Futoshiki systematicky dosazuje čísla do mřížky a hledá řešení splňující všechna pravidla hry (viz podsekcce 2.1). Postupuje přitom zleva doprava a shora dolů v rámci mřížky.

Při objevení prázdného pole zkouší algoritmus postupně všechna čísla v rozmezí od 1 do velikosti hrací mřížky. Po každém dosazení ověří, zda zadaná hodnota splňuje pravidla hry. Pokud ano, pokračuje k dalšímu poli. V případě, že žádné číslo z rozsahu pravidla nesplňuje, algoritmus se vrátí na předchozí pole a znovu zkouší jiná čísla. Tento proces se opakuje, dokud není nalezeno validní řešení nebo dokud nejsou vyzkoušeny všechny možné kombinace čísel.

2.3 Forward Checking

Forward checking [2] rozšiřuje základní princip systematického prohledávání Backtrackingu a zvyšuje jeho efektivitu tím, že snižuje počet zkoušených kombinací. Na začátku algoritmu se pro každé hrací pole vytvoří jeho doména, což je množina všech hodnot, které dané pole může nabývat bez porušení pravidel.

Algoritmus poté na základě zvolené strategie (heuristiky) vybere jedno z polí k vyplnění. Do tohoto pole je dosazena hodnota z jeho domény, načež jsou upraveny domény ostatních polí. Z těchto domén se vyloučí hodnoty, které by v kombinaci s dosazenou hodnotou porušily pravidla hry.

Tento postup se iterativně opakuje, přičemž algoritmus přechází k dalšímu poli, pokud jsou stále platné hodnoty v jeho doméně. V případě, že některé pole vyčerpá všechny možnosti (jeho doména se stane prázdnou), algoritmus se vrací zpět k předchozímu kroku (backtracking) a zvolí jinou hodnotu. Proces pokračuje, dokud není nalezeno platné řešení nebo dokud nejsou vyčerpány všechny možnosti.

2.3.1 Heuristiky Forward Checkingu

Heuristika určuje strategii výběru hracího pole a hodnoty, která bude dosazena do tohoto pole. Existují dvě hlavní kategorie heuristik: heuristiky pro výběr hracího pole a heuristiky pro výběr hodnoty z domény.

Heuristiky výběru hracího pole

Mezi heuristiky pro výběr hracího pole patří:

- Nejvíce omezená proměnná: Tato heuristika vybírá hrací pole, které má nejmenší doménu, tedy pole, pro které je nejvíce omezený výběr možných hodnot.
- Nejvíce omezující proměnná: Heuristika volí hrací pole, které má největší vliv na omezení ostatních polí, tedy pole, které ovlivňuje nejvíce ostatních hodnot v mřížce.

Heuristiky výběru hodnoty z domény

Mezi heuristiky pro výběr hodnoty z domény patří:

- Nejméně omezující hodnota: Tato heuristika vybírá hodnotu, která má nejmenší vliv na ostatní domény. Tímto způsobem se minimalizuje riziko, že se algoritmus dostane do slepé uličky, když později dojdou možnosti pro jiná pole.
- Náhodná hodnota: Tento přístup vybírá hodnotu z domény náhodně.

2.3.2 Variace Forward Checkingu

Uložení domén může probíhat několika způsoby. Mezi klasické způsoby patří ukládání hodnot do pole nebo seznamu, což však může být problematické, pokud je potřeba efektivně vylučovat hodnoty z domény nebo ověřovat, zda se hodnota v doméně nachází. Tento problém je způsoben nutností projít celé pole nebo seznam, což je časově náročné. Tento problém lze vyřešit pomocí hašovacích množin, které využívají hašovací tabulky. Ověření, zda se hodnota nachází v doméně, je efektivní s průměrnou časovou složitostí $O(1)$, a při mazání hodnoty není třeba přeskupovat prvky, na rozdíl od běžného pole.

S rostoucí velikostí mřížky se zvyšuje i využití paměti. Efektivnější způsob ukládání domén je možný pomocí bitmap, které vylepšují využití paměti tím, že doménu reprezentují jako jedno číslo místo celého pole čísel. V tomto případě každé číslo v doméně je reprezentováno jedním bitem. Bitmapy jsou velmi efektivní pro menší domény, ale s rostoucí velikostí domény se může velikost bitmapy stát problematickou, protože jsou omezeny velikostí čísla a počtem bitů, které mohou být použity.

Pro implementaci tohoto algoritmu jsem využil dva způsoby: pomocí hašovacích množin a bitmap. Implementace je podobná, u hašovacích množin se využívají funkce pro práci s množinami, zatímco u bitmap se používají bitové operace.

2.3.3 Forward Checking a Futoshiki

Forward Checking pro řešení Futoshiki nejprve vygeneruje domény pro každé hrací pole. Generování jsem implementoval tak, že nejprve zjistím, které hodnoty byly již v každém řádku a sloupci použity. Poté vytvořím doménu pro každé pole, která obsahuje všechny možné hodnoty, a odeberu hodnoty, které se vyskytly v daném řádku a sloupci, čímž zajistím, že nebudou opakovány. Dále projdu všechny nerovnosti a zajistím, aby hodnoty v doménách ovlivněných těmito nerovnostmi nebyly v rozporu s pravidly. Konkrétně z domény pole, které má být menší, odstraním hodnoty, které jsou větší nebo rovné maximální hodnotě sousedícího pole, a z domény pole, které má být větší, odstraním hodnoty menší nebo rovné minimální hodnotě sousedícího pole.

Po vygenerování domén počíná fáze zapisování hodnot. Pro zvolení hracího pole používám heuristiku nejvíce omezená proměnná, což znamená, že vybírám pole s nejmenší doménou. Z domény tohoto pole pak vyberu hodnotu podle heuristiky náhodné hodnoty. Po dosažení hodnoty upravím domény všech ovlivněných polí tak, aby stále platila pravidla. Tento postup se opakuje, dokud není nalezeno řešení nebo dokud nejsou vyzkoušeny všechny možné cesty.

2.4 Arc Consistency #3

Algoritmus Arc Consistency #3, známý také jako AC-3, rozšiřuje techniky snižování počtu zkoušených kombinací Forward Checkingu tím, že zajišťuje, aby během celého běhu algoritmu byly všechny domény konzistentní vzhledem ke všem omezením. Tento stav se nazývá Arc Consistency.

AC-3 pracuje iterativně s množinou takzvaných oblouků, kde každý oblouk reprezentuje vztah mezi dvěma proměnnými a jejich doménami. Na začátku algoritmu se všechny oblouky reprezentující omezení (například nerovnosti ve Futoshiki) zařadí do fronty. Algoritmus poté iterativně vybírá oblouky z fronty a provádí následující kroky:

1. Revize oblouku: Pro každý oblouk (X, Y) , kde X a Y jsou proměnné, se kontroluje, zda každá hodnota v doméně X má alespoň jednu odpovídající hodnotu v doméně Y , která splňuje omezení mezi X a Y . Pokud ne, hodnoty, které toto omezení porušují, jsou z domény X odstraněny.
2. Aktualizace fronty: Pokud se doména X změní, všechny oblouky ovlivněné X jsou přidány zpět do fronty pro další kontrolu.

Tento proces pokračuje, dokud není fronta prázdná, což znamená, že všechny domény jsou konzistentní s ohledem na daná omezení.

Na rozdíl od Forward Checkingu, který kontroluje konzistenci pouze lokálně při dosažení hodnoty, AC-3 zajišťuje globální konzistenci na úrovni všech proměnných a omezení.

2.4.1 AC-3 a Futoshiki

V implementaci AC-3 pro Futoshiki je nejprve nutné vygenerovat domény pro všechna hrací pole tak, aby splňovaly stav Arc Consistency. Tento proces zahrnuje odstranění hodnot, které porušují pravidla Futoshiki. Pro generování domén využívám stejný algoritmus jako u Forward Checkingu. Po vygenerování domén následuje fáze postupného dosazování hodnot.

Při zapisování hodnot jsou hrací pole a hodnota z jeho domény vybírány pomocí heuristik, stejně jako ve Forward Checkingu. Konkrétně používám heuristiku nejvíce omezená proměnná pro zvolení pole a heuristiku náhodná hodnota pro volbu hodnoty z domény zvoleného pole. Po dosažení hodnoty se upraví domény všech polí ve stejném řádku, sloupci a také všech polí ovlivněných nerovnostmi.

Pokud se doména některého pole změní, ověřují se všechny oblouky, ve kterých toto pole figuruje. V případě nekonzistence (výskyt hodnoty, která porušuje omezení) se doména druhého pole v oblouku upraví tak, aby konzistence byla obnovena. Tento proces jsem implementoval rekurzivně: jakmile dojde ke změně domény druhého pole, jsou znovu ověřeny všechny oblouky, kde toto pole figuruje. Tento postup se opakuje, dokud se žádná doména již nezmění, což znamená, že byl zajištěn stav Arc Consistency. Díky rekurzivní implementaci je odstraněna potřeba využití fronty, která je u klasické implementace AC-3 využita.

Kapitola 3

Existující formy konečných automatů

3.1 Konečný automat

Konečné automaty jsou základní matematické modely sloužící k popisu a analýze regulárních jazyků. Reprezentují abstraktní stroje, které zpracovávají vstupní řetězce symbolů a rozhodují, zda patří do jazyka, který automat popisuje. Rozlišujeme dva hlavní typy konečných automatů: deterministický konečný automat (DKA), v němž každý stav má jednoznačně definovaný přechod pro každý symbol, a nedeterministický konečný automat (NKA), jenž umožňuje více přechodů nebo i jejich absenci pro konkrétní symbol. Oba typy mají stejný vyjadřovací potenciál, protože každý nedeterministický konečný automat lze převést na ekvivalentní deterministický.

3.1.1 Deterministický konečný automat

Deterministický konečný automat je pětice $(Q, \Sigma, \delta, q_0, F)$, kde:

- Q je konečná neprázdná množina stavů.
- Σ je vstupní abeceda, konečná množina vstupních symbolů.
- δ je přechodová funkce ve tvaru $Q \times \Sigma \rightarrow Q$.
- q_0 je počáteční stav, kde $q_0 \in Q$.
- F je množina koncových nebo přijímacích stavů, kde $F \subseteq Q$.

3.1.2 Nedeterministický konečný automat

Nedeterministický konečný automat je pětice $(Q, \Sigma, \delta, q_0, F)$, kde:

- Q je konečná neprázdná množina stavů.
- Σ je vstupní abeceda, konečná množina vstupních symbolů.
- δ je přechodová funkce ve tvaru $Q \times \Sigma \rightarrow Q$, kde $\Sigma \cup \{\varepsilon\}$.
- q_0 je počáteční stav, kde $q_0 \in Q$.
- F je množina koncových nebo přijímacích stavů, kde $F \subseteq Q$.

3.1.3 Dvojcestný konečný automat

Dvojcestné konečné automaty rozšiřují klasické konečné automaty tím, že čtecí hlava se může pohybovat po vstupní pásce nejen zleva doprava, ale také zpět, tedy doprava i doleva. To jim umožňuje prozkoumávat části vstupu vícekrát. Stejně jako klasické konečné automaty se i dvojcestné konečné automaty dělí na deterministické a nedeterministické.

I když dvojcestné automaty působí výkonněji než jednocestné automaty, mají stejnou vyjadřovací sílu: přijímají pouze regulární jazyky. Tento výsledek vychází z faktu, že každý dvojcestný konečný automat může být převeden na ekvivalentní jednocestný automat (za cenu možného zvýšení počtu stavů).

Vstupní řetězec je ohraničen z obou stran speciálními symboly $\{ |-, -| \}$, které nejsou součástí vstupní abecedy Σ . Tyto symboly slouží k indikaci začátku a konce vstupu a čtecí hlava se za ně nesmí dostat. Pokud by čtecí hlava narazila na jeden z těchto symbolů, automat zůstane na stejném místě nebo přejde do koncového stavu.

Definice

Dvojcestné konečné automaty jsou definované jako osmice $(Q, \Sigma, |-, -|, \delta, s, t, r)$, kde:

- Q je konečná množina stavů.
- Σ je konečná množina vstupních symbolů.
- $|-$ je levý koncový symbol, přičemž platí $|- \notin \Sigma$.
- $-|$ je pravý koncový symbol, přičemž platí $-| \notin \Sigma$.
- δ je přechodová funkce ve tvaru $Q \times (\Sigma \cup \{|-, -|\}) \rightarrow (Q \times \{L, R\})$, přičemž L a R jsou směry pohybu čtecí hlavy.
- s je počáteční stav, kde $s \in Q$.
- t je akceptující stav, kde $t \in Q$.
- r je neakceptující stav, kde $r \in Q$.

3.1.4 Skákající automaty

Skákající automat umožňuje čtecí hlavě skákat po vstupní pásce. Vstupní páska je rozdělena do buněk, přičemž každá buňka obsahuje jeden symbol vstupního řetězce. Automat čte symbol z buňky, na kterou ukazuje čtecí hlava, a na základě aktuálního stavu a tohoto symbolu určuje, do jakého stavu přejde. Na rozdíl od klasických konečných automatů, které se pohybují po pásce pouze zleva doprava, může skákající automat přeskakovat mezi libovolnými buňkami. Pokud je symbol již přečten, nemůže být přečten znovu.

3.1.5 Skákající automat s opakovaným čtením

Definice

Skákající automat s opakovaným čtením lze definovat jako pěticu $M = (Q, \Sigma, R, sF)$, kde:

- Q je konečná množina stavů
- Σ je konečná množina vstupních symbolů

- $R \subseteq Q \times \Sigma^* \times Q$ je konečná ternární relace, nazývaná množinou pravidel
 - Každé pravidlo $(p, y, q) \in R$ je zapsáno jako $py \rightarrow q$
 - Jestliže je aktuální stav automatu q a na vstupu je symbol odpovídající symbolu y , automat přejde do stavu q
 - Vstupní ukazatel zůstane na stejné pozici a nepřečte vstupní symbol
- $s \in Q$ je počáteční stav
- $F \subseteq Q$ je množina konečných stavů

S podporou nerovností

TEMP: definice stejná jako u skáňícího automatu s opakovaným čtením. Změní se pouze množina pravidel:

- $R \subseteq Q \times C(\Sigma^*) \times Q$
 - C je predikát, který musí být splněn aby proběhl přechod
 - $p[c] \rightarrow q$
 - $p[x < 10] \rightarrow q$

Kapitola 4

Dvojdimenzionální automaty

4.1 Dvojdimenzionální skákající automat

Sedmice $(Q, \Sigma, \Delta, R_1, R_2, s, F)$, kde:

- Q je konečná množina stavů
- Σ je konečná množina vstupních symbolů
- Δ je množina povolených směrů
- R_1 množina pravidel pro pohyb ve tvaru $(Q \times \Sigma^* \times \Delta) \implies Q$
- R_2 množina pravidel pro skoky ve tvaru $(Q \times \Sigma^*) \implies Q$
 - Necht $a \in \Sigma$; $u, v \in \Sigma^*$. Pak $upayv \implies uaqyv$, právě když $py \implies q \in R$.
- $s \in Q$ je počáteční stav
- $F \subseteq Q$ je množina konečných stavů

Kapitola 5

Pravidla dvojdimenzionálního skákajícího automatu

V této kapitole se podrobně zaměříme na pravidla přechodů dvojdimenzionálního skákajícího automatu, který je navrhnut pro řešení hlavolamu Futoshiki.

5.1 Základní značení

- $##$ - hranice hrací mřížky
- $\#$ - hranice mezi hracími poli
- N - nerovnosti mezi poli
- F - vyplněné herní pole
- X - nevyplněné herní pole
- $\{X\}$ - pole s neprázdnou doménou
- $\{L\}$ - pole s nejmenší doménou
- $\{\}$ - pole s prázdnou doménou
- E - pole bez domény
- R, L, U, D - směry pohybu v mřížce

5.2 Vytvoření domén polí

Pravidla

Vytvoření domén řádků a sloupců

$Q_0\#\#R \Rightarrow Q_{01}$	Posun doprava pro vynechání okraje mřížky
$Q_{01}\#\#D \Rightarrow Q_{11}$	Posun dolů pro vynechání okraje mřížky
$Q_{10}\#\#R \Rightarrow Q_{19}$	Posun doprava a přesun do stavu ověření konce mřížky
$Q_{11}FR \Rightarrow Q_{11}$	Odstranění hodnoty z domény řádku a sloupce a posun doprava
$Q_{11}\#\#D \Rightarrow Q_{20}$	Posun dolů pokud hranice mřížky
$Q_{11}R \Rightarrow Q_{11}$	Jinak posun doprava
$Q_{19}\#D \Rightarrow Q_{11}$	Následuje další řádek, přesun do stavu úpravy domén
$Q_{19}U \Rightarrow Q_{39}$	Konec mřížky, přesun do stavu vytvoření domén polí
$Q_{20}\#\#L \Rightarrow Q_{29}$	Posun doleva a přesun do stavu ověření konce mřížky
$Q_{21}FL \Rightarrow Q_{21}$	Odstranění hodnoty z domény řádku a sloupce a posun doleva
$Q_{21}\#\#D \Rightarrow Q_{10}$	Posun dolů pokud hranice mřížky
$Q_{21}L \Rightarrow Q_{21}$	Jinak posun doleva
$Q_{29}\#D \Rightarrow Q_{21}$	Následuje další řádek, přesun do stavu úpravy domén
$Q_{29}U \Rightarrow Q_{49}$	Konec mřížky, přesun do stavu vytvoření domén polí

Vytvoření domén polí

$Q_{30}\#\#R \Rightarrow Q_{39}$	Posun doprava a přesun do stavu ověření konce mřížky
$Q_{31}ER \Rightarrow Q_{31}$	Vytvoření domény na základě domén řádku a sloupce
$Q_{31}\#\#U \Rightarrow Q_{40}$	Posun nahoru pokud hranice mřížky
$Q_{31}R \Rightarrow Q_{31}$	Jinak posun doprava
$Q_{39}\#L \Rightarrow Q_{31}$	Existuje předchozí řádek, přesun do stavu vytváření domén polí
$Q_{39} \Rightarrow Q_5$	Konec mřížky
$Q_{40}\#\#L \Rightarrow Q_{49}$	Posun doleva a přesun do stavu ověření konce mřížky
$Q_{41}EL \Rightarrow Q_{41}$	Vytvoření domény na základě domén řádku a sloupce
$Q_{41}\#\#U \Rightarrow Q_{30}$	Posun nahoru pokud hranice mřížky
$Q_{41}L \Rightarrow Q_{41}$	Jinak posun doleva
$Q_{49}\#U \Rightarrow Q_{41}$	Existuje předchozí řádek, přesun do stavu vytváření domén polí
$Q_{49} \Rightarrow Q_5$	Konec mřížky
$Q_5 \Rightarrow \text{TODO}$	pravidla pro nerovnosti

5.3 Řešení hry

Pravidla pro řešení hry

Pravidla

$Q_6\{L\} \Rightarrow Q_7$	Skok na pole s nejmenší doménou
----------------------------	---------------------------------

Kapitola 6

Implementace

Tato kapitola podrobně popisuje implementaci aplikace pro řešení hry Futoshiki. V rámci řešení jsou využity algoritmy uvedené v předchozích kapitolách. Také je detailně popsáno uživatelské prostředí aplikace, která je vytvořena jako terminálová aplikace, tedy veškeré prvky rozhraní jsou zobrazeny pomocí textu vytištěného do terminálu.

Aplikace je implementována v programovacím jazyce Rust za využití knihovny `termint`, která zjednodušuje tvorbu uživatelského rozhraní v terminálu. Vývoj probíhal na operačním systému Linux.

6.1 Spuštění aplikace

Při spouštění aplikace si může uživatel pomocí argumentů přizpůsobit nastavení hry, například zvolit algoritmus, který bude Futoshiki řešit, velikost mřížky nebo spustit benchmark.

Argumenty pro spuštění hry

- `-s`, `-size [číslo]`: Velikost mřížky (výchozí je 4)
- `-solver [typ]`: Algoritmus pro řešení Futoshiki (více popsáno později)
- `-t`, `-theme [téma]`: Téma aplikace
 - `light`: Světlé téma
 - `dark`: Tmavé téma
 - `default`: Výchozí téma

Argumenty pro benchmark

V případě že by chtěl uživatel spustit benchmark, musí využít argument `bench`, za kterým mohou následovat tyto argumenty:

- `-s`, `-size [číslo]`: Velikost testované mřížky (lze zadat vícekrát)
- `-solver [typ]`: Testovaný algoritmus (lze zadat vícekrát)
- `-r`, `-repeats [číslo]`: Počet opakování testu
- `-b`, `-boards [číslo]`: Počet testovaných mřížek

Dostupné typy algoritmů

- `bt`, `backtract`, `backtracking`: Backtracking
- `fc`, `forward-check`, `forward-checking`: Forward Checking
- `fcb`, `forward-check-bit`, `forward-checking-bit`: Forward Checking s bitovými operacemi
- `ac3`, `arc-cons3`, `arc-consistency3`: Arc Consistency #3
- `ac3b`, `arc-cons3-bit`, `arc-consistency3-bit`: Arc Consistency #3 s bitovými operacemi

Konfigurace aplikace

Pokud je aplikace spuštěna s argumentem `config`, otevře se konfigurační soubor ve výchozím textovém editoru. Jestliže soubor neexistuje, automaticky se vytvoří. Uživatel si v něm může nastavit výchozí hodnoty pro velikost mřížky, použitý algoritmus pro nalezení řešení a téma aplikace.

6.2 Vzhled a možnosti aplikace

Po spuštění hry se zobrazí mřížka Futoshiki, která má čtvercový tvar s předem nastavenou velikostí. V této mřížce je jedno z polí zvýrazněno, to značí právě označené pole. Pro změnu označeného pole lze využít klávesy šipek nebo klávesy `h j k l`, které využívá textový editor Vim pro navigaci v textu. Aplikaci lze kdykoliv ukončit stiskem kláves `Q` a `Esc`. Aplikace obsahuje tři obrazovky:

1. Vytváření mřížky
2. Řešení mřížky
3. Změna řešícího algoritmu

1	3		2
		v	
			<
4		v	
		>	

Obrázek 6.1: Vzhled aplikace po spuštění hry se světlým tématem

6.2.1 Vytváření mřížky

Tento režim umožňuje uživateli vytvořit vlastní mřížku, kterou lze následně nechat složit jedním z algoritmů.

Funkcionalita

1. Přidání hodnoty:

- Napsáním číselné hodnoty na klávesnici se hodnota přidá do označeného pole.
- Pokud hodnota přesáhne maximální povolenou hodnotu, odstraní se číslice z vyšších řádů.

2. Smazání hodnoty:

- Hodnotu lze smazat zapsáním čísla nula nebo klávesou **Delete**.

3. Přidání nerovnosti:

- Právě označené pole bude prvním prvek nerovnosti.
- Stisknutím klávesy **>** (větší) nebo **<** (menší) se vybere typ nerovnosti.
- Zvolením sousedního pole se nastaví druhý prvek nerovnosti a přidá se samotná nerovnost.

4. Smazání nerovnosti:

- Postup je stejný jako při přidávání nerovnosti, pouze místo kláves **>** nebo **<** se použije klávesa **C**.

Spuštění řešení

Po dokončení vytváření mřížky lze stisknutím klávesy **S** aktivovat aktuálně zvolený algoritmus, který následně mřížku vyřeší. V případě úspěchu se do mřížky doplní zbývající hodnoty a vypíše se zpráva, že mřížka byla složena.

6.2.2 Řešení mřížky

V tomto režimu může uživatel mřížku řešit manuálně.

Funkcionalita

1. Generování mřížky:

- Stisknutím klávesy **Enter** se vygeneruje náhodná mřížka.

2. Řešení mřížky:

- Hodnoty lze do polí zapisovat stejným způsobem jako při vytváření mřížky.
- Nelze přepisovat hodnoty, které byly vygenerovány a nelze měnit nerovnosti.

Dokončení řešení

Jestliže je mřížka úspěšně vyřešena, zobrazí se zpráva o úspěšném složení. Pokud má uživatel problém se složením mřížky, lze opět využít nastaveného algoritmu stisknutím klávesy **S** a tím zobrazit správné řešení.

6.2.3 Změna řešícího algoritmu

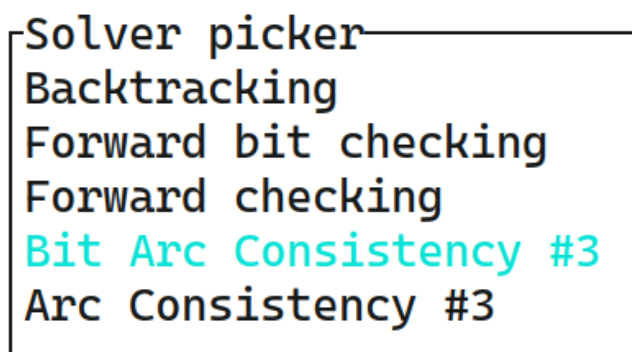
Uživatel má také možnost algoritmus použitý pro řešení Futoshiki změnit, aniž by musel hru spouštět znovu za pomoci odpovídajících argumentů.

1. Přejít mezi stránkou výběru a stránkou hry:

- Stisknutím klávesy **Tab** se přepne stránka.
- Pokud je aktuální stránka hry, přepne se na stránku výběru, v opačném případě se přejde na stránku hry.

2. Výběr algoritmu:

- Výběr lze měnit pomocí kláves šipka nahoru a šipka dolů.
- Pro potvrzení výběru je třeba stisknout klávesu **Enter**.



Obrázek 6.2: Stránka výběru řešícího algoritmu

Kapitola 7

Testování

Tato kapitola se zaměřuje na testování aplikace, především na ověření správnosti a výkonnosti implementovaných algoritmů. Testování zahrnuje kontrolu správnosti výsledků algoritmů, aby byla zajištěna jejich funkčnost při řešení hry Futoshiki. Současně je hodnocena časová náročnost algoritmů, přičemž jejich výkonnost je porovnána při různých velikostech hrací mřížky.

7.1 Správnost řešení

Při testování správnosti řešení se zaměřuji na několik klíčových částí aplikace: samotné algoritmy, ověřovač složení a proces generování mřížky.

Testování algoritmů

Pro testování algoritmů jsem vygeneroval čtyři různé mřížky s různými úrovněmi obtížnosti ze stránky futoshiki.com [1]. Každý algoritmus tyto mřížky vyřešil a výsledky byly následně zkontrolovány pomocí ověřovače složení, který zajišťuje, že výsledná mřížka splňuje pravidla Futoshiki.

Testování ověřovače složení

Ověřovač složení byl testován na mřížkách vygenerovaných ze stránky futoshiki.com [1], které byly předem složeny. Pro ověření správnosti jsem u některých mřížek změnil jejich řešení tak, aby obsahovaly chyby. Tímto způsobem jsem testoval schopnost ověřovače určit, zda je mřížka správně složena či nikoli.

Testování generování mřížky

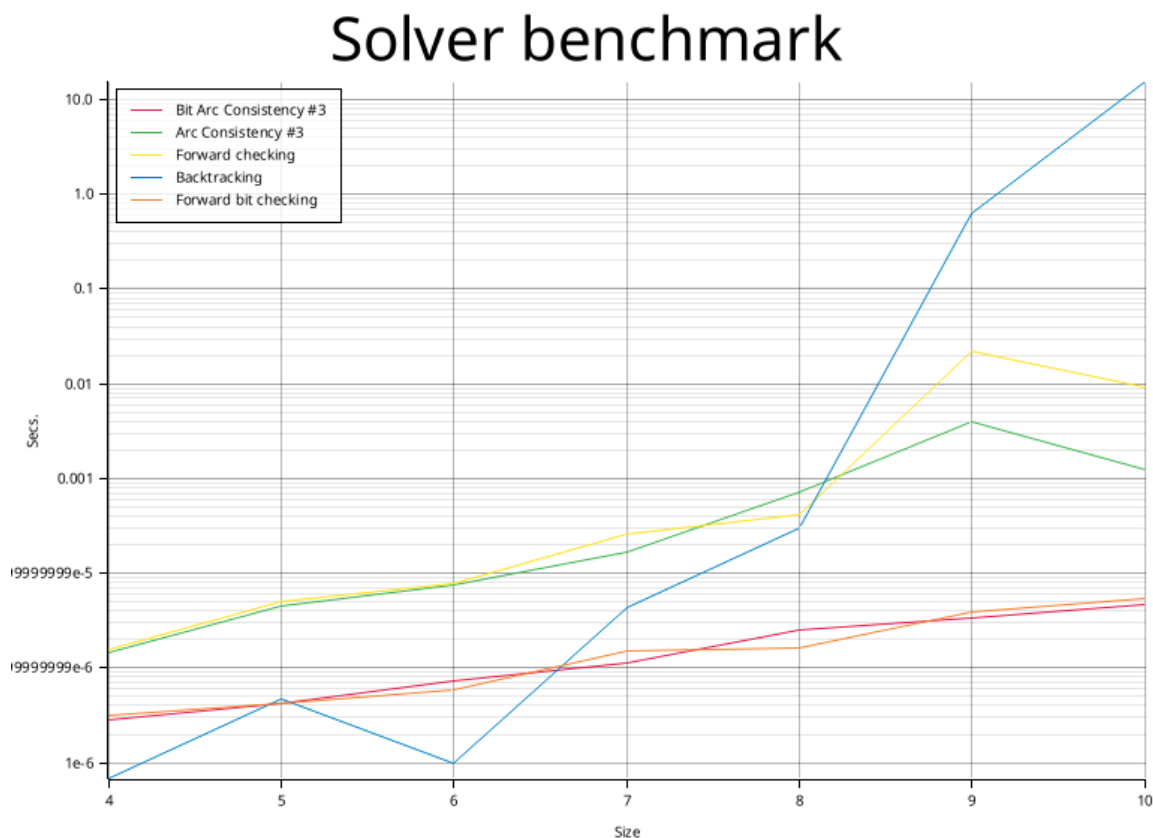
V testovacím procesu generování mřížek bylo nejdříve vytvořeno několik náhodných mřížek, které byly následně složeny pomocí jednoho z algoritmů. Výsledné složené mřížky byly zkontrolovány ověřovačem složení, což zajišťuje, že každá vygenerovaná mřížka je řešitelná a splňuje pravidla hry.

7.2 Časová náročnost

Testování časové náročnosti je prováděno po spuštění aplikace s argumentem **bench**. Tento režim umožňuje měřit výkonnost zvolených algoritmů (při výchozím nastavení všech algoritmů) na různých velikostech mřížek.

Pro každou velikost mřížky je náhodně vygenerována mřížka, kterou následně vyřeší každý algoritmus. Aby se minimalizoval vliv náhodných výkyvů, je test opakován předem nastaveným počtem iterací (výchozí hodnota je 10). Pro každou velikost mřížky je navíc test proveden na nastaveném počtu mřížek (výchozí hodnota je 1). Po dokončení testování aplikace zobrazí v terminálu výsledky obsahující nejlepší, průměrný a nejhorší naměřený čas pro každý algoritmus a velikost mřížky.

Kromě textového výstupu aplikace automaticky generuje graf, který vizualizuje průměrné časy jednotlivých algoritmů pro různé velikosti mřížek. Tento graf usnadňuje porovnání efektivity algoritmů při řešení s různě velkými mřížkami.



Obrázek 7.1: Srovnání časové náročnosti všech algoritmů při různých velikostech mřížky (4×4 až 10×10).

7.2.1 Zhodnocení

Jak ukazuje graf, menší mřížky zvládnou všechny algoritmy rychle. Backtracking je však nejrychlejší a to díky své jednoduchosti a absenci režie, jako je generování domén polí. S rostoucí velikostí mřížky však jeho výkon výrazně klesá, což je dáno jeho systematickým

prohledáváním všech možností, kdy počet možností, které je třeba vyzkoušet, exponenciálně roste.

Na větších mřížkách jsou rozdíly mezi algoritmy výraznější. Je také důležité zmínit, že osa y je logaritmická, což může zkreslovat reálně rozdíly. Backtracking je při velikosti 10×10 pomalejší přibližně o deset sekund než další nejpomalejší algoritmus. Nejlépe si vedly algoritmy Forward Checking a Arc Consistency #3 a to v jejich implementaci s bitmapami.

Kapitola 8

Závěr

Literatura

- [1] DASCALU, V. *Futoshiki Online Puzzles*. 2024.
- [2] ING. FRANTIŠEK ZBOŘIL, P. doc. *Přednášky předmětu Základy umělé inteligence*. 2024.
- [3] WIKIPEDIE, P. *Futoshiki*. 23. března 2009.

Příloha A

Obsah CD

- `src/`: adresář obsahující zdrojové soubory a testy
- `xsleza26.pdf`: text bakalářské práce
- `doc/`: adresář se zdrojovými soubory pro vytvoření PDF souboru s textem bakalářské práce
- `README.md`: základní popis a návod spuštění aplikace